Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann
Date: July 20, 2026, Date of submission: 16. August 2026

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.
Darmstadt

TECHNISCHE UNIVERSITÄT DARMSTADT

CCPS Control and Cyber-Physical Systems

Electrical Engineering and Information Technology Department

Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.

Date: July 20, 2026
Date of submission: 16. August 2026

Darmstadt

Technische Universität Darmstadt
Institut für Automatisierungstechnik und Mechatronik
Fachgebiet Control and Cyber-Physical Systems
Prof. Dr.-Ing. Rolf Findeisen

Contents

1 Abstract **1**

2 Introduction **2**

3 Optimal Control and Lyapunov Stability Theory **3**
3.1 Discrete-Time Dynamical Systems and Stability . . . 3
3.1.1 Lyapunov Stability . . . 3
3.2 Model Predictive Control and Stability . . . 4
3.2.1 Problem Formulation . . . 4
3.2.2 Terminal Ingredients for Stability . . . 4
3.2.3 Closed-Loop Stability . . . 6
3.3 Stability without Terminal Constraints . . . 6

4 Foundations of Neural Control Policies **7**
4.1 Artificial Neural Networks . . . 7
4.1.1 Multi-Layer Perceptrons . . . 7
4.1.2 Optimization and Training . . . 7
4.2 Adversarial Training and Falsification . . . 7

5 Formal Verification of Neural Control Policies **8**
5.1 Mixed-Integer Linear Programming . . . 8
5.2 Branch and Bound . . . 8
5.3 $\alpha\beta$-CROWN . . . 8

6 Learning Lyapunov-Stable Imitation Controllers **9**
6.1 Neural Policy Approximation and Imitation Learning . . . 9
6.1.1 Data Generation and MPC Expert . . . 9
6.1.2 Policy Network Architecture and Imitation Objective . . . 9
6.2 Lyapunov Learning for Stability Certification . . . 9
6.2.1 Lyapunov Network Architecture . . . 10
6.2.2 Loss Formulation . . . 10
6.2.3 Falsification-Guided Training . . . 13
6.2.4 ρ-Sublevel Estimation . . . 14
6.3 Formal Verification Methodology . . . 14

7 Experimental Evaluation and Certification Results **15**
7.1 System Modeling and Problem Formulation . . . 15
7.1.1 Double Integrator . . . 15
7.1.2 Cartpole . . . 15

7.2 Linear System Results: Double Integrator 16
7.3 Nonlinear System Results: Cartpole 16

8 Discussion **17**
8.1 Qualitative Discussion of Results 17
8.2 Challenges with Nonlinear Dynamics 17
8.3 Scalability and Limitations 17

9 Conclusion **18**

A Appendix **19**

1 Abstract

2 Introduction

3 Optimal Control and Lyapunov Stability Theory

3.1 Discrete-Time Dynamical Systems and Stability

To implement an optimal control scheme on digital hardware, the underlying continuous-time system dynamics

$$\dot{x}(t) = f(x(t), u(t)) \tag{3.1}$$

must be discretized, where $x(t) \in \mathcal{X}$ and $u(t) \in \mathcal{U}$ denote the continuous-time state and input vectors, respectively. In this work, the corresponding discrete-time system

$$x_{k+1} = f_d(x_k, u_k) \tag{3.2}$$

is obtained by approximating the continuous dynamics using a numerical integration scheme with a constant sampling time T_s, such as the forward Euler or a fourth-order Runge-Kutta (*RK4*) method **something**.

3.1.1 Lyapunov Stability

Let the origin $(x, u) = (\mathbf{0}, \mathbf{0})$ be an equilibrium point of the system such that $f_d(\mathbf{0}, \mathbf{0}) = \mathbf{0}$. Suppose the system is controlled by a state-feedback control policy $u_k = \kappa(x_k)$. The closed-loop system is said to be *Lyapunov stable* if there exists a positive definite Lyapunov function $V(x_k)$ such that:

$$\begin{aligned} V(\mathbf{0}) &= 0 \\ V(x_k) &> 0 \qquad \forall x_k \in \mathcal{X} \setminus \{\mathbf{0}\} \\ V(f_d(x_k, \kappa(x_k))) - V(x_k) &\leq 0 \qquad \forall x_k \in \mathcal{X} \setminus \{\mathbf{0}\}. \end{aligned} \tag{3.3}$$

Furthermore, the equilibrium point is *locally asymptotically stable* if the Lyapunov function additionally satisfies a strictly negative definite decrease condition along the closed-loop trajectories:

$$V(f_d(x_k, \kappa(x_k))) - V(x_k) < 0 \qquad \forall x_k \in \mathcal{X} \setminus \{\mathbf{0}\}. \tag{3.4}$$

Asymptotic stability is typically the desired property for control systems, as it ensures that the system will converge to the equilibrium point over time.

3.2 Model Predictive Control and Stability

This section provides a brief overview of the Model Predictive Control (*MPC*) framework and its stability properties. First, the general MPC problem is formulated, followed by a discussion on the terminal set and terminal cost constraints. Afterwards, the closed-loop stability of the MPC scheme is analyzed.

3.2.1 Problem Formulation

For the nominal MPC setup considered in this work, the following finite-horizon optimal control problem is solved at each time step k given the current system state x_k:

$$\begin{aligned} \min_{\{x_i\}_{i=0}^{N},\{u_i\}_{i=0}^{N-1}} \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + V_f(x_N) \\ \text{s.t.} \quad & x_{i+1} = f_d(x_i, u_i), && i = 0, \dots, N-1, \\ & x_i \in \mathcal{X}, && i = 0, \dots, N, \\ & u_i \in \mathcal{U}, && i = 0, \dots, N-1, \\ & x_0 = x_k. \end{aligned} \tag{3.5}$$

Here, N denotes the prediction horizon, $\ell(x_i, u_i)$ is the stage cost function, and $V_f(x_N)$ represents the terminal cost function. The system dynamics are represented by the discrete-time function $f_d(x_i, u_i)$, while $\mathcal{X}$ and $\mathcal{U}$ denote the state and input constraint sets, respectively.

A quadratic cost function is used for the stage cost, defined as

$$\ell(x_i, u_i) = x_i^\top Q x_i + u_i^\top R u_i, \tag{3.6}$$

where $Q \succeq 0$ and $R \succ 0$ are the state and input weighting matrices, respectively. Following the receding horizon principle, only the first control input u_0^* from the optimal sequence is applied to the system, and the optimization is repeated at the next sampling instance.

3.2.2 Terminal Ingredients for Stability

To ensure stability of a closed-loop system of a nominal MPC, a terminal set $\mathcal{X}_f \subseteq \mathcal{X}$ and a terminal cost function $V_f(x_N)$ are introduced. The terminal set $\mathcal{X}_f$ is a positively invariant set for the system under a local control law $u = \mu(x_k)$, and the terminal cost $V_f(x_N)$ is a Lyapunov function for the system within this set. However, to include these terminal ingredients, Eq. (3.5) must be augmented with the following terminal constraint:

$$x_N \in \mathcal{X}_f. \tag{3.7}$$

The property of a positive invariant set $\mathcal{X}_f$ is defined as a set of states where a state $x_k \in \mathcal{X}_f$ remains in $\mathcal{X}_f$ for all future time steps under the local control law $\mu(x_k)$,

$$f_d(x, \mu(x)) \in \mathcal{X}_f \quad \forall x \in \mathcal{X}_f. \tag{3.8}$$

This predictive mechanism is visualized in Section 3.2.2. While the initial state x_0 and the intermediate predicted states x_1, x_2 are only restricted to the safe state space $\mathcal{X}$, the terminal constraint Eq. (3.7) forces the final predicted state x_N to terminate strictly inside the invariant terminal set $\mathcal{X}_f$. Together with the terminal cost $V_f(x_N)$, which acts as a local Lyapunov function inside $\mathcal{X}_f$, these ingredients establish a mathematical bridge.

For nonlinear systems, the terminal set $\mathcal{X}_f$ is typically computed using linearized system dynamics around the equilibrium point $(x, u) = (\mathbf{0}, \mathbf{0})$, yielding the linear discrete-time state-space model:

$$x_{k+1} = Ax_k + Bu_k. \tag{3.9}$$

A widely used choice for these terminal ingredients is then derived from the Linear-Quadratic Regulator (*LQR*). Here, the local control law $\mu(x_k)$ is chosen as a linear state-feedback policy $\mu(x_k) = -Kx_k$, where the optimal LQR gain K is given by

$$K = \left(R + B^\top PB\right)^{-1} B^\top PA. \tag{3.10}$$

The matrix $P \succ 0$ is obtained as the unique positive definite solution to the discrete-time Algebraic Riccati Equation (*DARE*):

$$P = A^\top PA - A^\top PB \left(R + B^\top PB\right)^{-1} B^\top PA + Q. \tag{3.11}$$

This matrix parameterizes the quadratic terminal cost function $V_f(x_N) = x_N^\top P x_N$. Accordingly, the terminal set $\mathcal{X}_f$ is defined as an ellipsoidal sub-level set of this cost function:

$$\mathcal{X}_f = \left\{ x \in \mathbb{R}^n \;\middle|\; x^\top Px \leq \alpha \right\}. \tag{3.12}$$

To satisfy the positive invariance condition Eq. (3.8) for the original nonlinear system, the scaling parameter $\alpha > 0$ must be chosen sufficiently small. This ensures not only that all states within $\mathcal{X}_f$ satisfy the state constraints $\mathcal{X}$ and input constraints $\mathcal{U}$, but also that the local quadratic Lyapunov descent condition dominates the higher-order linearization errors of the nonlinear dynamics.

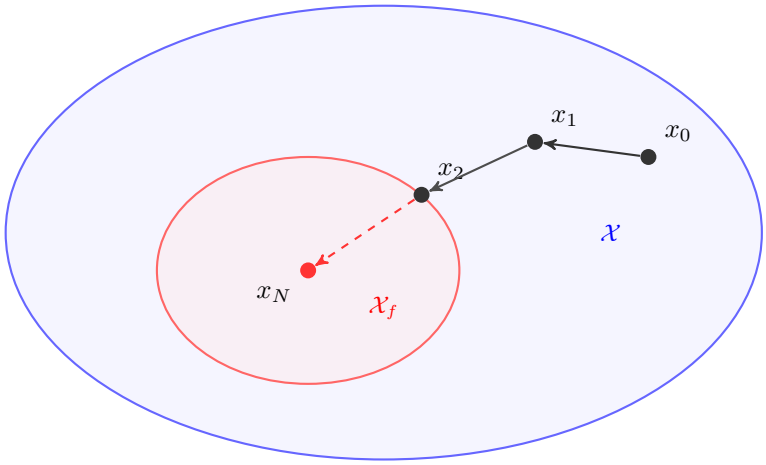

Figure 3.1: Comparison of the terminal set $\mathcal{X}_f$ (red) with the entire state space $\mathcal{X}$ (blue).

3.2.3 Closed-Loop Stability

TODO: ADD FORMULATION FOR RELAXED MPC STABILITY CONDITION

$$V_N(f_d(x_k, \mu(x_k))) \leq V_N(x_k) - \alpha_N \cdot \ell(x_k, \mu(x_k)), \quad \forall x_k \in \mathcal{X}_f \tag{3.13}$$

3.3 Stability without Terminal Constraints

only for no terminal constraints -> not asymptotically stable, but can be shown to be practically stable

$$\alpha_N = 1 - \frac{(\gamma - 1)^N}{\gamma^{N-1} - (\gamma - 1)^N} \tag{3.14}$$

if $\alpha_N > 0$ then

$$N < 2 + \frac{\ln(\gamma - 1)}{\ln(\gamma) - \ln(\gamma - 1)}, \tag{3.15}$$

then holds

$$V_N(x_k) \leq \gamma \ell^*(x_k) \tag{3.16}$$

4 Foundations of Neural Control Policies

4.1 Artificial Neural Networks

4.1.1 Multi-Layer Perceptrons

4.1.2 Optimization and Training

4.2 Adversarial Training and Falsification

5 Formal Verification of Neural Control Policies

5.1 Mixed-Integer Linear Programming

5.2 Branch and Bound

5.3 $\alpha\beta$-CROWN

6 Learning Lyapunov-Stable Imitation Controllers

6.1 Neural Policy Approximation and Imitation Learning

6.1.1 Data Generation and MPC Expert

6.1.2 Policy Network Architecture and Imitation Objective

$$\pi_\theta(x) = \min\left(\max\left(f_\theta(x) - f_\theta(x^*) + u^*, u_{\min}\right), u_{\max}\right) \tag{6.1}$$

Policy Network Loss $S_x = \mathrm{diag}(x_{\text{scale}})$ $S_u = \mathrm{diag}(a_{\text{scale}})$

$$d(x) = \frac{1}{n_x} \left\| S_x^{-1}(x - x^*) \right\|_1 . \tag{6.2}$$

$$w_x(x) = w_{\min} + (1 - w_{\min}) \left(\frac{1}{1 + d(x)} \right)^\alpha , \tag{6.3}$$

$$\mathcal{L}_x = \frac{1}{M n_u} \sum_{x \in \mathcal{X}_\pi} w_x(x) \left\| S_u^{-1} \big(\pi_V(x) - \pi_0(x) \big) \right\|_2^2 . \tag{6.4}$$

$$\mathcal{L}_f = \frac{1}{M n_x} \sum_{x \in \mathcal{X}_\pi} \left(\left\| \left[x_{k+1} - \bar{x} \right]_+ \right\|_1 + \left\| \left[\underline{x} - x_{k+1} \right]_+ \right\|_1 \right) . \tag{6.5}$$

$$\mathcal{L}_\pi = \omega_x \mathcal{L}_x + \omega_f \mathcal{L}_f , \tag{6.6}$$

6.2 Lyapunov Learning for Stability Certification

To achieve formal stability with learned controllers, a Lyapunov candidate is trained to certify stability. A falsification-guided framework is used to find stability violations and therefore make the Lyapunov candidate a valid certificate for stability.

6.2.1 Lyapunov Network Architecture

Ensuring that the learned Lyapunov candidate is positive definite is crucial for stability certification. To achieve this, a neural network architecture is designed to guarantee positive definiteness by construction, following the approach in **lyapunov-stable-nn**. The architecture combines the flexibility of neural networks while ensuring the mathematical properties required for Lyapunov functions. Resulting from this design, the Lyapunov candidate is defined as:

$$V_\theta(x_k) = |f_\theta(x_k) - f_\theta(x^*)| + \left\|(\epsilon I + R^\top R)(x_k - x^*)\right\|_1, \tag{6.7}$$

where $f_\theta : \mathbb{R}^n \to \mathbb{R}$ is a neural network, $x_k \in \mathbb{R}^n$ is the current state, $x^* \in \mathbb{R}^n$ is the equilibrium, $\epsilon > 0$ is a small regularization constant, and $R \in \mathbb{R}^{n \times n}$ is a learnable matrix. While the first term is only positive semi-definite and may vanish away from the equilibrium, the second term guarantees strict positive definiteness $\forall x_k \in \mathcal{X} \setminus \{x^*\}$, as the matrix $(\epsilon I + R^\top R)$ is strictly positive definite by construction for any $\epsilon > 0$.

6.2.2 Loss Formulation

To ensure that the Lyapunov candidate described in Eq. (6.7) can be certified for stability, the loss function must comprise multiple components. It utilizes the ReLU function $[\cdot]_+$ for specific penalties, alongside a weighted mean to aggregate the sample losses. Central to the following loss terms is the estimation of the region of attraction (ROA). Throughout the training process, a target level ρ is considered, leading to the corresponding ρ-sublevel set within the training domain $\mathcal{B}$, defined as

$$\mathcal{V}_\rho = \{x \in \mathcal{B} \mid V(x) \leq \rho\}. \tag{6.8}$$

To prevent this certified region from trivially collapsing towards the origin during optimization, and to ensure numerical stability during normalizations, ρ is strictly constrained by a lower bound $\rho_{\text{min}} > 0$. Furthermore, the next states x_{k+1} needed during loss computation are evaluated on-the-fly using the system dynamics and the policy $u_k = \pi_V(x_k)$, which may be actively learning during the co-training process.

Relative Lyapunov Decrease Loss is intended to ensure that the Lyapunov candidate decreases along the system trajectories. This is achieved by minimizing the relative difference between the Lyapunov candidate evaluated at the current state and the next state. For this purpose, the per-sample Lyapunov decrease term is defined as

$$e_V = V(x_{k+1}) - (1 - \kappa) \cdot V(x_k), \tag{6.9}$$

where $\kappa \in (0, 1)$ is a hyperparameter that controls the desired rate of decrease. To balance the optimization scale across different magnitudes of V, we normalize this violation to obtain the relative decrease loss:

$$\mathcal{L}_{\text{dec}} = \left[\frac{e_V}{\max(|V(x_k)|, \epsilon_{\text{rel}})}\right]_+, \tag{6.10}$$

where $\epsilon_{\text{rel}} > 0$ prevents division by zero. However, the normalization of the decrease can also be omitted, depending on the settings.

Relative State Invariance Loss penalizes trajectories that escape the training state space $\mathcal{B} = \{x \in \mathbb{R}^n \mid \underline{x} \leq x \leq \bar{x}\}$. To evaluate the relative one-step state-constraint violation, the scaling matrix $S_{\text{range}} = \text{diag}(\bar{x} - \underline{x})$ is utilized. The loss per sample for the next state x_{k+1} is computed as

$$\mathcal{L}_{\text{inv}} = \left\| S_{\text{range}}^{-1} \left[x_{k+1} - \bar{x} \right]_+ \right\|_1 + \left\| S_{\text{range}}^{-1} \left[\underline{x} - x_{k+1} \right]_+ \right\|_1 . \tag{6.11}$$

Combined ρ-Gated Condition Loss is the sample-wise ρ-gated combination of the relative Lyapunov decrease loss from Eq. (6.10) and the relative state invariance loss from Eq. (6.11),

$$\mathcal{L}_{\text{cond}} = \mathcal{L}_{\text{dec}} + \omega_{\text{inv}} \cdot \mathcal{L}_{\text{inv}}, \tag{6.12}$$

where ω_{inv} is a hyperparameter balancing the relative importance of the two loss components. However, to apply the ρ-gate appropriately, $\mathcal{L}_{\text{cond}}$ is weighted for each sample using the soft sublevel weight

$$w_\rho = \sigma \left(s \cdot \frac{\rho - V(x)}{\max(\rho, \epsilon_{\text{rel}})} \right), \tag{6.13}$$

where $s > 0$ denotes the gate sharpness. This smooth formulation ensures that the weights inside $\mathcal{V}_\rho$ are ≈ 1, whereas the weights for states far outside of ρ approach 0. Evaluating these terms for each sample $x \in \mathcal{X}_\rho$, the weighted mean of the combined condition loss and the ρ-gating weight results in the final loss formulation:

$$\mathcal{L}_{\rho\text{-gated}} = \frac{\sum_{x \in \mathcal{X}_\rho} w_\rho \mathcal{L}_{\text{cond}}}{\sum_{x \in \mathcal{X}_\rho} w_\rho}. \tag{6.14}$$

ROA Loss tries to enlarge the region of attraction by uniformly drawing a batch of M samples $\mathcal{X}_{\text{ROA}} \subseteq \mathcal{B} \setminus \xi\mathcal{B}$ from the shell of the training space. The loss is then evaluated over these candidates using the following formulation:

$$\mathcal{L}_{\text{ROA}} = \frac{1}{M} \sum_{x \in \mathcal{X}_{\text{ROA}}} \ln \left(\left[\frac{V(x)}{\rho} - 1 \right]_+ + 1 \right). \tag{6.15}$$

LIRPA-Condition Loss makes the Lyapunov function more verifier-friendly by including formal violations of the Lyapunov decrease condition as defined in Eq. (6.9), which are identified using the LIRPA framework **lirpabook**. In order to compute these violations efficiently, only a filtered subset of regions is evaluated. This subset contains regions for which either the center x_{center}, lower bound x_{lower} or upper bound x_{upper} lies within $\mathcal{V}_\rho$. LIRPA then provides a formal lower bound $e_{V,\text{lower}}$ on the signed Lyapunov decrease $-e_V$ for each of these regions. After evaluation, the LIRPA-condition violation for each region is computed as

$$\mathcal{M}_{\text{L}} = \frac{\ln([-e_{V,\text{lower}}]_+ + 1)}{\max(V(x_{\text{center}}), \epsilon_{\text{rel}})}. \tag{6.16}$$

The final LIRPA-condition loss is then computed as the weighted mean over all evaluated regions, where the weight is defined using an exponential function of the squared Euclidean distance between the region center x_{center} and the equilibrium x^*:

$$w_L = \exp\left(-\alpha_{\text{L}} \cdot \|x_{\text{center}} - x^*\|_2^2 \right), \tag{6.17}$$

where $\alpha_{\mathrm{L}} > 0$ is a hyperparameter controlling the decay rate of the weight. Finally, the LIRPA-condition loss over N_{L} regions is computed as

$$\mathcal{L}_{\mathrm{L}} = \frac{\sum_{i=1}^{N_{\mathrm{L}}} w_L^{(i)} \cdot \mathcal{M}_{\mathrm{L}}^{(i)}}{\sum_{i=1}^{N_{\mathrm{L}}} w_L^{(i)}}. \tag{6.18}$$

Lyapunov Scaling Loss is introduced to ensure that the Lyapunov candidate does not collapse to 0. This is achieved by drawing a batch of M samples $\mathcal{X}_V \subseteq \mathcal{B}$, using a quasi-Monte-Carlo sequence. The logarithm of the mean Lyapunov value over this batch is then defined as: $\ell_V = \ln\left(\frac{1}{M}\sum_{x\in\mathcal{X}_V} V(x)\right)$, leading to the scaling loss formulation

$$\mathcal{L}_V = \left(\ell_{V,0} - \ell_V\right)^2. \tag{6.19}$$

Note that the reference value $\ell_{V,0}$ is determined by computing ℓ_V using the initial Lyapunov candidate. Furthermore, to prevent the Lyapunov candidate from overfitting to a fixed set of points, a new batch of samples $\mathcal{X}_V$ is drawn with a specified probability at each optimization step.

Policy Scaling Loss encourages the learned policy π_V to remain close to the output of the original imitation controller π_0, which is based on the MPC teacher. A batch of M samples $\mathcal{X}_\pi \subseteq \mathcal{B}$ is generated using a quasi-Monte-Carlo sequence. To prevent overfitting, this batch is redrawn after a specified number of optimization steps. The deviation between the learned policy and the imitation controller is evaluated using a normalized squared difference. The loss is then formulated as

$$\mathcal{L}_\pi = \frac{\sum_{x\in\mathcal{X}_\pi} \|\pi_V(x) - \pi_0(x)\|_2^2}{\sum_{x\in\mathcal{X}_\pi} \|\pi_0(x)\|_2^2} \tag{6.20}$$

R-Matrix Loss tries to prevent the collapse of the R matrix in Eq. (6.7) towards a norm of 0 during training. To achieve this, the Frobenius norm of the R matrix is computed and compared to a target value $\ell_{R,0}$

$$\mathcal{L}_R = \left(\ell_{R,0} - \|R\|_F\right)^2. \tag{6.21}$$

Note that $\ell_{R,0} = \|R\|_F$ is automatically set based on the initial R matrix.

Final Lyapunov Loss represents the combined objective function, summing all previously defined weighted components, given by

$$\mathcal{L}_{\text{total}} = \omega_{\text{cond}}\mathcal{L}_{\rho\text{-gated}} + \omega_{\text{ROA}}\mathcal{L}_{\text{ROA}} + \omega_{\mathrm{L}}\mathcal{L}_{\mathrm{L}} + \omega_V\mathcal{L}_V + \omega_\pi\mathcal{L}_\pi + \omega_R\mathcal{L}_R, \tag{6.22}$$

where the weights ω are hyperparameters that balance the relative importance of the individual loss components.

6.2.3 Falsification-Guided Training

Relying solely on uniform state sampling is not sufficient to reliably detect violations of the Lyapunov condition, especially in higher-dimensional state spaces. To overcome this problem, falsification-guided training actively searches for adversarial counterexamples and incorporates them into the training distribution. This is made possible by an adversarial replay buffer that reconciles the exploration of the estimated attractor region with the targeted exploitation of known error modes.

PGD Falsification is utilized to identify counterexamples that violate the Lyapunov conditions and iteratively inject them into the counterexample pool $\mathcal{C}$. Specifically, an L_∞-Projected Gradient Descent (PGD) is employed to find adversarial states $x_{\text{adv}} \in \mathcal{B}$ that maximize the condition violation defined in Eq. (6.12) and Eq. (6.13). The per-sample PGD objective is defined as

$$J_{\text{PGD}} = -w_\rho \mathcal{L}_{\text{cond}}. \tag{6.23}$$

The adversarial search is initialized using the explorative pool $\mathcal{S}$, with up to 25% of the batch supplemented by existing counterexamples to encourage local refinement. The algorithm minimizes J_{PGD} by iteratively updating the input states along the sign of their local gradient with a constant step size α, followed by a projection back onto the training bounds $\mathcal{B}$. To retain the strongest violations and mitigate the effects of overshooting, the state yielding the minimum objective value across all K iterations is preserved for each candidate. Ultimately, only states strictly within $\mathcal{V}_\rho$ that exceed the violation tolerance and lie outside the origin exclusion region are retained as counterexamples.

Adversarial Replay Buffer manages the sample distribution for the ρ-gated condition loss $\mathcal{L}_{\rho\text{-gated}}$. It maintains two distinct state pools: an explorative pool $\mathcal{S}$ and a counterexample pool $\mathcal{C}$.

To focus the optimization on relevant regions, the explorative pool $\mathcal{S}$ is periodically repopulated. Every outer epoch, candidate states are oversampled uniformly from the training domain $\mathcal{B}$. The final pool is then constructed using a probabilistic sampling strategy that prioritizes states within an expanded sublevel boundary $m \cdot \rho$, controlled by a scalar margin $m > 1$. As outlined in Algorithm 1, a continuous sampling probability is computed via a sigmoid function, assigning high weights to states inside the estimated region of attraction and exponentially decaying weights to states further outside.

Algorithm 1: Probabilistic Explorative State Pool Resampling

Input: Lyapunov function V_θ, current ROA estimate ρ, margin m, sharpness s, target size N_x
Output: Updated state pool $\mathcal{S}$ of size N_x

1 Generate $4N_x$ candidates: $\mathcal{X} \sim \mathcal{U}(\mathcal{B})$;

2 For all $x \in \mathcal{X}$, assign weight $w(x) \leftarrow \sigma\left(s \, \frac{m \cdot \rho - V_\theta(x)}{\max(\rho, 10^{-9})}\right) + \epsilon$;

3 $\mathcal{S} \leftarrow$ Sample N_x states from $\mathcal{X}$ with probability $P(x) \propto w(x)$;

The counterexample pool $\mathcal{C}$ stores the critical violations identified during the PGD falsification step. To maintain the relevance of the adversarial states and prevent overfitting to outdated violations, a time-to-live mechanism is implemented. Each injected counterexample is assigned an initial age counter $\tau = 0$, which is incremented upon every subsequent falsification phase. Once a counterexample reaches the

maximum age $\tau_{\max}$, it is removed from the buffer. This guarantees that the condition loss continuously penalizes recent violations while allowing the network to gracefully discard resolved edge cases as the Lyapunov candidate evolves.

6.2.4 ρ-Sublevel Estimation

To compute the loss, particularly the ρ-gated condition loss in Eq. (6.14), the target level ρ, which bounds the sublevel set $\mathcal{V}_\rho$, must be continuously estimated during training. Since the goal is to find the largest level set of the Lyapunov candidate $V_\theta(x)$ that lies entirely within the defined training bounds $\mathcal{B}$, a level set can only escape the safe region by intersecting its boundary $\partial\mathcal{B}$. Therefore, the maximum certifiable ρ is mathematically strictly determined by the minimum value of the Lyapunov candidate at its boundary. According to the formulation proposed by **yang2024lyapunov**, it is sufficient to evaluate $V_\theta(x)$ exclusively on the boundary $\partial\mathcal{B}$ rather than inefficiently sampling the entire state space, in order to estimate the bounds of the ROA. However, since the boundary is a high-dimensional surface and the neural network parameterized in $V_\theta(x)$ forms a highly non-convex landscape, simple uniform sampling on the boundary is insufficient to find the true minimum. Therefore, a Projected Gradient Descent (PGD) approach is employed to actively search for the worst-case boundary points. Starting from uniformly sampled points $x_\partial \sim \mathcal{U}(\partial\mathcal{B})$, PGD iteratively minimizes $V_\theta(x_\partial)$ by taking steps along the negative gradient and projecting the updated states strictly back onto the boundary surfaces. To actively encourage the region of attraction to expand over the course of the co-training process, the formally certifiable limit is intentionally overestimated, as suggested in **yang2024lyapunov**. Specifically, a target ρ is computed by applying a growth factor $\gamma > 1$ to the identified boundary minimum identified by the PGD:

$$\tilde{\rho} = \max\left(\rho_{\min}, \gamma \cdot \min_{x\in\partial\mathcal{B}} V_\theta(x)\right). \tag{6.24}$$

By targeting a value for $\tilde{\rho}$ that is slightly above the currently safe maximum, the formulation deliberately leads to small violations near the boundary. These violations are then captured by the aforementioned adversarial replay buffer, effectively forcing the optimization to continuously expand the sublevel set outward. Finally, to ensure numerical stability and prevent irregular jumps in the loss landscape caused by fluctuating local minima across different batches, the ρ value actually used for gating is updated using an exponential moving average (EMA):

$$\rho^{(t)} = \beta_{\text{EMA}}\rho^{(t-1)} + (1-\beta_{\text{EMA}})\tilde{\rho}, \tag{6.25}$$

where $\beta_{\text{EMA}} \in [0, 1)$ is the decay rate.

6.3 Formal Verification Methodology

7 Experimental Evaluation and Certification Results

7.1 System Modeling and Problem Formulation

For our experimental setup, two benchmark systems are selected: the double integrator and the cartpole. These systems provide a good balance between simplicity and complexity allowing us to certify the stability of the closed-loop system.

7.1.1 Double Integrator

The double integrator is a simple second-order linear system; therefore, it serves as the ideal baseline system in the learning and certification pipeline. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.1$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{DI},d}(x_k, u_k) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ \Delta t \end{bmatrix} u_k \tag{7.1}$$

where $x_k = \begin{bmatrix} p_k & v_k \end{bmatrix}^\top$ is the state vector containing the position and velocity of the system, respectively, and u_k denotes acceleration.

The *MPC setup* is formulated according to the problem in Eq. (3.5) subject to the double-integrator dynamics in Eq. (7.1). A prediction horizon of $N = 20$ steps is chosen, and the states and inputs are penalized using the weighting matrices $Q = \text{diag}(15, 1)$ and $R = 0.1$.

7.1.2 Cartpole

The inverted pendulum on a cart, commonly referred to as the cartpole system, is used as a nonlinear benchmark system in this work. There are four states in the system: the cart position p, the cart velocity v, the pendulum angle θ, and the pendulum angular velocity ω. Here, the state vector is defined as $x = \begin{bmatrix} p & v & \theta & \omega \end{bmatrix}^\top$, and the input u is the force F applied to the cart. This leads to the following continuous-time dynamics of the cartpole system:

$$f_{\text{CP}}(x,u) = \begin{bmatrix} \dot{p} \\ \dot{v} \\ \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v \\ \frac{F+ml\omega^2\sin(\theta)-mg\sin(\theta)\cos(\theta)}{M+m-m\cos^2(\theta)} \\ \omega \\ \frac{-F\cos(\theta)-ml\omega^2\sin(\theta)\cos(\theta)+(M+m)g\sin(\theta)}{l(M+m-m\cos^2(\theta))} \end{bmatrix} \tag{7.2}$$

where $M = 1\,\text{kg}$ is the mass of the cart, $m = 0.1\,\text{kg}$ is the mass of the pendulum, $l = 0.8\,\text{m}$ is the length of the pendulum, and $g = 9.81\,\text{m/s}^2$ is the acceleration due to gravity. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.05\,\text{s}$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{CP},d}(x_k, u_k) = x_k + \Delta t f_{\text{CP}}(x_k, u_k) \tag{7.3}$$

For the nonlinear benchmark, a prediction horizon of $N = 40$ stages is chosen. The system states and control inputs are penalized within the optimization framework of Eq. (3.5) using the weighting matrices $Q = \text{diag}(10, 1, 10, 0.01)$ and $R = 0.5$, respectively, subject to the dynamics defined in Eq. (7.3).

7.2 Linear System Results: Double Integrator

7.3 Nonlinear System Results: Cartpole

8 Discussion

8.1 Qualitative Discussion of Results

8.2 Challenges with Nonlinear Dynamics

8.3 Scalability and Limitations

9 Conclusion

A Appendix